



AI LABS / DESIGN SYSTEM

Technical documentation

Architecture / APIs / Tokens / Packaging / Operations

Angular components, tokens, and the component showcase

Repository baseline 0.1.2 | Commit 88d2cc8

22 September 2026

DESIGN > CODE > REVIEW > RELEASE > ADOPT

A source-checked reference for designers, developers, and maintainers.

Includes practical limits, verification steps, and official workflow references.

Contents

Click a section title to jump to it. PDF bookmarks provide the same navigation.

1. Purpose and scope	3
2. System at a glance	3
3. Repository structure	4
4. Prerequisites and installation	5
5. Using the showcase	6
6. Consuming components in Angular	7
7. Component API reference	8
8. Design tokens and theming	15
9. Application internals	16
10. Accessibility guidance	17
11. Building and deployment	17
12. Extending the design system	18
13. Quality assurance	19
14. Security and privacy	20
15. Troubleshooting	20
16. Known limitations and roadmap priorities	21
17. Maintenance checklist	21
18. Installation schematic and package contract	22
19. Integration details and implementation boundaries	22
20. Design-to-release operating model	23
Appendix A. Public exports	23
Appendix B. Configuration reference	24
Appendix C. Verification and source references	25

Application version: 0.1.2 **Framework:** Angular 22.1 **Documentation revision:** 22 September 2026 **Repository:** ItsVivekRamachandran/Ailabs-DesignSystem

Source baseline: commit 88d2cc8 **Companion:** docs/STEP_BY_STEP_GUIDE.md and the Step-by-Step Guide PDF cover Figma, installation, GitHub, npm releases, upgrades, and recovery.

The repository package name is AI Labs Design System (ailabs-design-system). The showcase header currently displays IGT & Everi; these labels refer to the same checked-in application. This reference describes local source behavior, not verified npm publication or Figma library status.

1. Purpose and scope

AI Labs Design System is a browser-based design-system showcase and component playground for AI-oriented products. It provides:

- CSS design tokens for color, spacing, radius, stroke, typography, shadow, and focus treatment.
- Standalone Angular components for common controls, navigation, search, date/time entry, and toolbars.
- Light and dark themes based on semantic token remapping.
- An interactive documentation site with component examples, token reference, live controls, and copyable Angular markup.

This document is intended for:

- **Application users**, who want to browse the design system and use the playground.
- **Angular developers**, who want to reuse or extend the components.
- **Design-system maintainers**, who need to understand the token and styling architecture.
- **Release engineers**, who build and deploy the static site.

Packaging status: The repository contains both the showcase application and an Angular Package Format library. `npm run build` creates the installable package in `dist/ailabs-design-system`, and `npm run pack` creates a distributable `.tgz` archive.

2. System at a glance

2.1 Technology stack

Area	Technology	Repository version
Application framework	Angular	22.1.7
Build system / CLI	@angular/build, Angular CLI	22.1.8
Language	TypeScript	6.0.x
Reactive utilities	Angular signals, RxJS	RxJS 7.8.2
Runtime integration	Zone.js	0.16.x
Styling	Global CSS and CSS custom properties	Native browser CSS
Rendering	Client-side Angular application	Static output

There is no server-side API, database, authentication layer, analytics SDK, or remote data source in the current application.

2.2 High-level architecture

```
Browser
|
+-- index.html
|
+-- src/main.ts
|
+-- bootstraps standalone AppComponent
|
+-- documentation/showcase template
+-- Angular design-system components
+-- TypeScript token metadata
+-- global CSS token layer
+-- component CSS layer
+-- showcase layout CSS
+-- local SVG search assets
```

The application is deliberately small and uses no router, dependency-injection services, NgModules, or HTTP client. AppComponent is the composition root. Each design-system component is standalone and uses ChangeDetectionStrategy.OnPush.

2.3 Runtime data flow

1. src/main.ts imports Zone.js and bootstraps AppComponent.
2. Angular renders the single documentation page from app.component.html.
3. AppComponent imports token metadata from tokens.ts to render color, spacing, radius, and stroke references.
4. Components receive values through Angular inputs and emit user changes through Angular outputs.
5. The theme signal sets data-theme on the root HTML element. Semantic CSS variables then resolve to light or dark values.
6. The chosen theme is stored in browser localStorage under ailabs-theme.
7. An IntersectionObserver tracks the currently visible documentation section.
8. Clipboard actions use navigator.clipboard to copy install text, color values, or playground markup.

3. Repository structure

```

.
|-- angular.json           Angular build and serve configuration
|-- index.html            Host HTML document and metadata
|-- package.json          Package metadata, peer dependencies, and scripts
|-- package-lock.json     Reproducible npm dependency graph
|-- ng-package.json       Angular library packaging configuration
|-- tsconfig.json         Strict TypeScript/Angular compiler settings
|-- tsconfig.app.json     Application compilation entry point
|-- tsconfig.lib*.json    Library compilation configurations
|-- docs/
|   |-- TECHNICAL_DOCUMENTATION.md  Maintainer and user manual
|-- output/pdf/              Distributable PDF documentation
`-- src/
    |-- main.ts              Browser bootstrap
    |-- public-api.ts        Public package exports
    |-- styles.css           Showcase/page layout and responsive rules
    |-- assets/search/       Legacy source SVG references used by the showcase
    |-- design-system/
    |   |-- tokens.css       Runtime CSS custom properties and themes
    |   |-- tokens.ts        Typed token metadata used by the showcase
    |   |-- components.css   Shared styles for all ds-* components
    `-- app/
        |-- app.component.ts  Page state and browser interactions
        |-- app.component.html Documentation page and interactive playground
        `-- design-system/
            |-- index.ts      Public TypeScript barrel exports
            `-- *.component.ts Standalone Angular components

```

Generated library files are written to `dist/ailabs-design-system/`; the showcase application is written to `dist/playground/browser/`. Generated files should not be edited manually.

4. Prerequisites and installation

4.1 Required software

Angular CLI 22.1.8 declares support for:

- Node.js ^22.22.3, ^24.15.0, or >=26.0.0.
- npm 8 or newer (the CLI also lists legacy compatible ranges).

The documentation environment used Node.js 24.16.0 and npm 11.13.0. The engine range above was read from the installed CLI package. Consult the [Angular compatibility reference](#) before changing framework versions.

4.2 Local setup

```

git clone https://github.com/ItsVivekRamachandran/Ailabs-DesignSystem.git
cd Ailabs-DesignSystem
npm ci
npm start

```

Open `http://localhost:4200/`. Angular's development server watches source files and reloads the page after changes.

Use `npm ci` in continuous integration and for clean reproducible installs. Use `npm install` when intentionally changing dependencies and updating `package-lock.json`.

4.3 Available commands

Command	Purpose
<code>npm start</code>	Start the development server (ng serve).
<code>npm run dev</code>	Alias for the development server.
<code>npm run build</code>	Build the installable Angular library.
<code>npm run pack</code>	Build the library and create a .tgz archive in dist/.
<code>npm run build:demo</code>	Create an optimized showcase application build.
<code>npm run build:pages</code>	Build with /Ailabs-DesignSystem/ as the base URL for GitHub Pages.
<code>npm run watch</code>	Continuously rebuild the library using the development configuration.
<code>npm run watch:demo</code>	Continuously rebuild the showcase application.

No repository scripts currently exist for unit tests, end-to-end tests, linting, formatting, or Storybook.

5. Using the showcase

5.1 Page navigation

The page has six tracked sections:

- Overview
- Foundations
- Colors
- Variables
- Components
- Playground

Selecting a navigation link updates the URL fragment without reloading the page and smoothly scrolls to the corresponding section. On initial load, a recognized URL fragment selects that section. While scrolling, `IntersectionObserver` updates the active navigation state.

5.2 Theme selection

Use the header theme control to switch between light and dark modes. The application:

- Updates the `data-theme` attribute on `<html>`.
- Updates the browser `theme-color` meta tag.
- Saves light or dark to `localStorage` as `ailabs-theme`.
- Defaults to light mode when the saved value is absent, invalid, or inaccessible.

The app does not currently derive the initial theme from `prefers-color-scheme`.

5.3 Component playground

The playground supports Button, Chip, Badge, Checkbox, Switch, Radio, Text field, Tabs, Menu item, List item, Search, and Toolbar. Choose a component in the left navigation and adjust its available controls. The preview and Angular markup update from signals and computed values.

The "Copy code" action writes the displayed markup to the clipboard and shows a temporary confirmation. Browser clipboard access may require HTTPS or localhost and user permission.

Date Calendar, Time Clock, Card, Tooltip, and Search Panel are demonstrated in the component catalog but are not selectable in the current playground menu.

6. Consuming components in Angular

Install the package through Angular CLI so its ng-add schematic can register the required global stylesheet automatically:

```
ng add ailabs-design-system
```

When targeting one application in a multi-project workspace, pass `--project <application-name>`. Restart a running development server after the workspace configuration changes.

6.1 Import pattern

All components are standalone and are exported from the `ailabs-design-system` package.

```
import { Component } from '@angular/core'
import {
  ButtonComponent,
  CheckboxComponent,
  InputComponent,
  ToolbarComponent,
} from 'ailabs-design-system'

@Component({
  selector: 'app-example',
  standalone: true,
  imports: [ButtonComponent, CheckboxComponent, InputComponent, ToolbarComponent],
  template: `
    <ds-input
      label="Workspace name"
      placeholder="Enter a name"
      [(value)]="workspaceName"
    />

    <ds-checkbox [(checked)]="rememberMe">Remember me</ds-checkbox>

    <button dsButton (click)="save()">Save</button>
  `,
})
export class ExampleComponent {
  workspaceName = ''
  rememberMe = false

  save(): void {
    // Application behavior belongs to the consumer.
  }
}
```

Angular's two-way binding syntax works where a component exposes an input and a corresponding `...Change` output, such as `value/valueChange` or `checked/checkedChange`.

6.2 Required global styles

Component templates rely on global CSS classes and variables. `ng add` registers the combined stylesheet automatically. When using `npm install` directly, add it to the consuming application's `angular.json` manually:

```
{
  "styles": [
    "ailabs-design-system/styles.css",
    "src/styles.css"
  ]
}
```

The package also exposes `ailabs-design-system/tokens.css` and `ailabs-design-system/components.css` separately when an application needs explicit ordering or customization. The consumer's own `src/styles.css` remains application-specific.

Because component styles are global, class names use the `ds-` prefix to reduce collisions. There is no Shadow DOM or Angular style encapsulation around the shared CSS.

6.3 Asset paths

Search icons are embedded in the component templates, so consumers do not need to configure asset copying for those components. Playground snippets can contain SVG symbol references defined only in the showcase, including `#icon-sparkles` and toolbar symbols. Supply equivalent symbols or replace those icons in the consuming app.

7. Component API reference

7.1 Button

Import: `ButtonComponent` **Selector:** `button[dsButton]`

Input	Type	Default	Description
<code>variant</code>	<code>'primary' 'secondary' 'outline' 'ghost'</code>	<code>'primary'</code>	Visual hierarchy.
<code>size</code>	<code>'sm' 'md' 'lg'</code>	<code>'md'</code>	Control dimensions and text size.
<code>iconOnly</code>	<code>boolean</code>	<code>false</code>	Produces square icon-button sizing.

```
<button dsButton variant="secondary" size="lg">Generate</button>
<button dsButton [iconOnly]="true" aria-label="Add item">...</button>
```

Use the native `disabled`, `type`, `click` events, and ARIA attributes directly on the button. Icon-only buttons require an accessible name.

7.2 Badge

Import: `BadgeComponent` **Selector:** `ds-badge`

Input	Type	Default
<code>tone</code>	<code>'neutral' 'violet' 'success' 'warning'</code>	<code>'neutral'</code>

Input	Type	Default
dot	boolean	false

```
<ds-badge tone="success" [dot]="true">Ready</ds-badge>
```

Badge content is projected. The optional dot is decorative and hidden from assistive technology.

7.3 Card

Import: CardComponent **Selector:** ds-card

Input	Type	Default
eyebrow	string	' '
title	string	' '
description	string	' '

```
<ds-card eyebrow="Insights" title="Model health" description="Updated just now">
  <p>Projected card content</p>
</ds-card>
```

The header is omitted when all three header inputs are empty.

7.4 Checkbox

Import: CheckboxComponent **Selector:** ds-checkbox

Input / output	Type	Default / payload
checked	boolean	false
indeterminate	boolean	false
disabled	boolean	false
checkedChange	EventEmitter<boolean>	New checked value

```
<ds-checkbox [(checked)]="includeInsights">Include insights</ds-checkbox>
```

A user change clears indeterminate, updates the internal input value, and emits checkedChange. Each instance creates a unique input ID.

7.5 Chip

Import: ChipComponent **Selector:** button[dsChip]

Input	Type	Default
variant	'filled' 'outlined'	'filled'
selected	boolean	false

Input	Type	Default
iconOnly	boolean	false

```
<button dsChip [selected]="active">AI ready</button>
```

The component provides styling only; consumers own selection state and click behavior. Supply `aria-pressed` when a chip acts as a toggle.

7.6 Text field

Imports: `InputComponent` or its export alias `TextFieldComponent` **Selectors:** `ds-input`, `ds-text-field`

Input / output	Type	Default / payload
label	string	''
hint	string	''
placeholder	string	''
type	string	'text'
icon	string	''; only 'search' renders an icon
value	string	''
error	string	''
disabled	boolean	false
valueChange	<code>EventEmitter<string></code>	Current input text

```
<ds-input
  label="Project name"
  hint="Visible to collaborators"
  placeholder="Enter project name"
  [(value)]= "projectName"
/>
```

When `error` is non-empty, it replaces the hint, sets `aria-invalid`, and links the input to the error text with `aria-describedby`.

7.7 Switch

Import: `SwitchComponent` **Selector:** `ds-switch`

Input / output	Type	Default / payload
checked	boolean	false
disabled	boolean	false
checkedChange	<code>EventEmitter<boolean></code>	New checked value

The internal checkbox uses `role="switch"`. Prefer a switch for settings that take effect immediately.

7.8 Radio

Import: RadioComponent **Selector:** ds-radio

Input / output	Type	Default / payload
name	string	Generated group-like name
value	string	' '
checked	boolean	false
disabled	boolean	false
checkedChange	EventEmitter<boolean>	Always true after user selection

```
<ds-radio name="model" value="fast" [checked]="model === 'fast'">Fast</ds-radio>
<ds-radio name="model" value="smart" [checked]="model === 'smart'">Smart</ds-radio>
```

Always provide the same explicit name for every option in a group. The component emits selection but not its value; the parent should associate the handler with the intended option.

7.9 Tab

Import: TabComponent **Selector:** button[dsTab]

Input	Type	Default
active	boolean	false

The host receives `role="tab"` and `aria-selected`. Place tabs in a `role="tablist"` container. The consumer must implement active-state changes, arrow-key navigation, `aria-controls`, and associated tab panels when building a complete tabs widget.

7.10 Tooltip

Import: TooltipComponent **Selector:** ds-tooltip

Input	Type	Default
text	string	' '
position	'top' 'bottom' 'left' 'right'	'top'
visible	boolean	false

Projected trigger content receives `aria-describedby` through a wrapper. CSS displays the bubble on hover, focus within the component, or when `visible` is true. Consumers should also test keyboard focus and touch behavior for their usage context.

7.11 Menu item

Import: MenuItemComponent **Selector:** button[dsMenuItem]

Input	Type	Default
shortcut	string	' '
active	boolean	false

The host receives `role="menuitem"`. Use inside an element with `role="menu"`. The consumer is responsible for menu opening/closing, roving focus, keyboard navigation, and action handling.

7.12 List item

Import: ListItemComponent **Selector:** button[dsListItem]

Input	Type	Default
title	string	' '
supportingText	string	' '
selected	boolean	false

The component has projection slots selected by [avatar], [title], and [trailing].

```
<button dsListItem title="AI workspace" supportingText="Updated moments ago">
  <span avatar>AW</span>
  <span trailing aria-hidden="true"></span>
</button>
```

7.13 Search bar

Import: SearchBarComponent **Selector:** ds-search-bar

Input / output	Type	Default / payload
placeholder	string	'Hinted search text'
ariaLabel	string	'Search'
value	string	' '
state	'default' 'hovered' 'focused'	'default'
valueChange	EventEmitter<string>	Current query
search	EventEmitter<string>	Submitted query
clear	EventEmitter<void>	Clear action

state is a showcase/visual-state control; actual browser hover and focus CSS also applies. Form submission prevents page navigation and emits the query.

7.14 Search panel

Import: SearchPanelComponent **Selector:** ds-search-panel

Input / output	Type	Default / payload
configuration	'input' 'supporting'	'input'
value	string	''
placeholder	string	'Hinted search text'
ariaLabel	string	'Search'
resultsLabel	string	'Search results'
results	readonly SearchResult[]	Built-in demo results when empty
valueChange	EventEmitter<string>	Current query
search	EventEmitter<string>	Submitted query
clear	EventEmitter<void>	Clear action
back	EventEmitter<void>	Back action
voice	EventEmitter<void>	Voice action
resultSelect	EventEmitter<SearchResult>	Selected result object

```
interface SearchResult {  
  readonly id: string  
  readonly label: string  
  readonly supportingText: string  
}
```

Pass real results explicitly in production. An empty array intentionally displays placeholder demo content rather than an empty state.

7.15 Date picker field

Import: DatePickerComponent **Selector:** ds-date-picker

Input / output	Type	Default / payload
label	string	'Label'
placeholder	string	'MM / DD / YYYY'
value	string	''
error	string	''
disabled	boolean	false
valueChange	EventEmitter<string>	Unparsed input string

This component is a styled text field. It does not parse, validate, localize, or open `DateCalendarComponent` automatically.

7.16 Date calendar

Import: `DateCalendarComponent` **Selector:** `ds-date-calendar`

Input / output	Type	Default / payload
view	'calendar' 'months' 'years'	'calendar'
selectedDay	number	24
selectedYear	number	2024
selectedDayChange	EventEmitter<number>	Selected current-month day

The present calendar is a fixed March demonstration: month labels, the 42-day grid, header text, and year list are static. Previous/next buttons have labels but no navigation handler. Month selection returns to the calendar view; year selection changes the year locally and returns to calendar view. Treat it as a visual prototype until full date logic is implemented.

7.17 Time picker field

Import: `TimePickerComponent` **Selector:** `ds-time-picker`

Its API mirrors the date field: `label`, `placeholder`, `value`, `error`, `disabled`, and `valueChange`. The default placeholder is `HH : MM`. Values are emitted as unparsed strings.

7.18 Time clock

Import: `TimeClockComponent` **Selector:** `ds-time-clock`

Input	Type	Default
mode	'dial' 'input'	'dial'
view	'hours' 'minutes'	'hours'
hour	string	'09'
minute	string	'30'
period	'AM' 'PM'	'AM'
error	string	' '

The dial calculates twelve item positions at 30-degree intervals. Selecting an item changes the component's internal hour or minute. The component currently exposes no change outputs, and input mode displays labels rather than editable time inputs. It is therefore a visual/interaction prototype, not a complete form control.

7.19 Toolbar

Import: ToolbarComponent **Selector:** ds-toolbar

Input	Type	Default
variant	'docked' 'floating-horizontal' 'floating-vertical'	'docked'
tone	'standard' 'vibrant'	'standard'
expanded	boolean	true
ariaLabel	string	'Page actions'

Content projection uses [toolbarLeading], the default slot, and [toolbarTrailing]. Leading and trailing groups always appear for the docked variant; floating variants hide them when collapsed. The internal container uses role="toolbar" and sets its orientation from the variant.

8. Design tokens and theming

8.1 Token layers

The token system has three conceptual layers:

1. **Primitive tokens** hold stable raw colors such as --primitive-primary-500.
2. **Semantic tokens** express intent such as --text-primary, --surface-default, or --border-focus.
3. **Stable component aliases** provide concise shared names such as --color-ink, --space-4, --shadow-md, and --focus-ring.

Components should prefer semantic or stable aliases over primitive colors. This allows a theme to change meaning without rewriting component CSS.

8.2 Color system

Primitive ramps include:

- Primary: 11 steps, 50 through 950.
- Secondary: 11 steps, 50 through 950.
- Support Gold: 11 steps, 50 through 950.
- Support Sand: 11 steps, 50 through 950.
- Neutral: 12 steps, 0 through 950.
- Black and White: 2 absolute values.

The TypeScript metadata describes 58 primitive color entries and 51 semantic color names across Background, Surface, Text, Primary action, Secondary action, Accent, Feedback, Icon, and Border groups.

8.3 Foundation scales

Scale	Values
Spacing	23 steps: 0, 1, 2, 3, 4, 6, 8, 10, 11, 12, 14, 16, 20, 24, 32, 40, 48, 56, 64, 80, 96, 120, 160 px
Radius	13 values from 0 px through 24 px, plus 100 px pill and 9999 px full
Stroke	0, 0.6667, 0.75, 1, 1.5, 2, 3, and 4 px
Font size	11 through 72 px across 13 named steps
Font weight	400, 500, 600, 700, and 800

8.4 Adding or changing a token

1. Add or change the CSS custom property in `src/design-system/tokens.css`.
2. For a theme-sensitive semantic token, define it in both `:root` and `:root[data-theme='dark']`.
3. If the documentation should enumerate it, update the matching metadata in `src/design-system/tokens.ts`.
4. Replace hardcoded component values with the semantic token where appropriate.
5. Inspect both themes and keyboard focus states.
6. Run the production build.

`tokens.css` and `tokens.ts` are maintained manually and are not generated from one another. Changes can drift unless both representations are reviewed.

9. Application internals

9.1 State model

AppComponent uses Angular signals for theme, current documentation section, copy feedback, selected playground component, and component controls. Computed values derive:

- The visible Light/Dark theme label.
- The selected playground component definition.
- Copyable Angular markup for the active playground state.

No application state leaves the browser. Theme preference is the only persisted value.

9.2 Browser APIs

API	Use	Fallback behavior
localStorage	Persist theme	Exceptions are caught; session still works.
IntersectionObserver	Active section tracking	No explicit fallback; navigation still scrolls when clicked.
History API	Replace URL fragment	Used without navigation/reload.

API	Use	Fallback behavior
Clipboard API	Copy values and code	No error UI is currently implemented.
<code>scrollIntoView</code>	Smooth section navigation	Depends on browser support and motion settings.

9.3 Change detection

All components use `OnPush`. Inputs, outputs, user events, and Angular signals provide the relevant update boundaries. When adding mutable object inputs, prefer replacing the object or array reference so `OnPush` consumers update predictably.

9.4 Styling and responsiveness

The site uses global responsive CSS at 900 px and 600 px breakpoints; individual component adaptations occur at 520 px. The app honors `prefers-reduced-motion`: `reduce` in the showcase stylesheet. Modern CSS features include custom properties, `color-mix()`, `:has()`, `clamp()`, grid, and flexbox, so target browsers should be current evergreen releases.

10. Accessibility guidance

The implementation includes useful foundations: native form controls, linked labels, unique IDs, focus-visible styling, error relationships, `aria-invalid`, roles for tabs/menu/toolbar/search/listbox, and reduced-motion CSS.

Consumers must still complete widget-level behavior:

- Add accessible names to every icon-only button.
- Manage tab activation, focus movement, panels, and arrow-key behavior.
- Manage menu focus, escape handling, and open/close state.
- Set `aria-pressed` on selectable chips where appropriate.
- Announce asynchronous search status and empty states.
- Validate color contrast after changing tokens.
- Test keyboard-only, screen-reader, zoom, forced-color, reduced-motion, and touch use.
- Do not treat placeholders as labels.

The repository does not currently include automated accessibility tests or a published WCAG conformance statement.

11. Building and deployment

11.1 Library build

```
npm ci
npm run build
```

Output is written to:

```
dist/ailabs-design-system/
```

This output follows Angular Package Format and contains the FESM bundle, type declarations, package manifest, README, and exported stylesheets. Create an installable archive with:

```
npm run pack
```

The resulting archive is written to `dist/ailabs-design-system-<version>.tgz`.

11.2 GitHub Pages build

```
npm run build:pages
```

This builds the showcase with the base href `/Ailabs-DesignSystem/`, matching the configured live demo path. The existing `.github/workflows/deploy-pages.yml` runs on pushes to `main` or manual dispatch. It uses Node 22, runs `npm ci` and `npm run build:pages`, copies `index.html` to `404.html`, uploads `dist/playground/browser/`, and deploys through the `github-pages` environment. The workflow grants contents: read, pages: write, and id-token: write; concurrency group pages cancels older in-progress runs.

Set repository Settings > Pages > Source to GitHub Actions. The workflow does not publish npm or synchronize Figma, and it contains no library-build or test job. See the companion guide for the complete commit, pull request, deployment, and release sequence and [GitHub Pages configuration](#).

For deployment at a different subpath, replace the base href:

```
npx ng build ailabs-design-system --base-href /your-subpath/
```

For a root-domain showcase deployment, use `npm run build:demo`.

11.3 Static hosting requirements

- Serve `index.html`, hashed JS/CSS bundles, and the `assets/` directory together.
- Use the correct base URL for subpath hosting.
- Serve over HTTPS if clipboard behavior is required outside localhost.
- Long-cache hashed assets; avoid permanently caching `index.html`.
- No server-side rewrite is currently required because the app has no Angular routes.

12. Extending the design system

12.1 Add a component

1. Create `src/app/design-system/<name>.component.ts`.
2. Use a `ds-` element selector or a `dsName` attribute selector.
3. Make the component standalone and use `ChangeDetectionStrategy.OnPush`.
4. Use semantic tokens and `ds-` prefixed classes in `components.css`.
5. Prefer native elements and semantics over recreated controls.
6. Add the export to `src/app/design-system/index.ts`.
7. Import the component in `AppComponent` if it will appear in the showcase.
8. Add a catalog example and, where appropriate, a playground definition and controls.
9. Document inputs, outputs, slots, keyboard interaction, and limitations.
10. Add tests before treating the component as production-ready.

12.2 API conventions

- Inputs use simple, explicit values and defaults.
- Two-way-bindable properties pair property with propertyChange.
- Native button attribute components preserve native events and attributes.
- Projected content provides labels and flexible icon/slot content.
- Host classes encode variants and state for global CSS.
- Component types are exported when consumers need them.

12.3 Publishing the library

The Angular library target is ready for local archive installation. Before publishing it to a registry:

1. Confirm ownership and availability of the ailabs-design-system npm name, or change it to an owned scope.
2. Add a license, changelog, and release policy.
3. Update the semantic version and document breaking changes.
4. Run a clean install, package build, archive installation test, and showcase build.
5. Publish only the contents generated in dist/ailabs-design-system.

13. Quality assurance

13.1 Current verification

The repository currently relies on strict compilation and manual visual/interaction review. TypeScript enables strict mode, strict templates, strict injection parameters, noImplicitReturns, and related safety flags.

Run before every merge:

```
npm ci
npm run build
npm run build:demo
```

Then manually check:

- Light and dark themes.
- Desktop and mobile widths.
- Keyboard focus for every control.
- Every playground component and copy action.
- Search clear, submit, back, voice, and result events.
- Form error and disabled states.
- Date and time prototype interactions.
- GitHub Pages base-path asset loading.

13.2 Recommended test strategy

Layer	Recommended coverage
Unit	Defaults, host classes, input changes, output payloads, state methods.

Layer	Recommended coverage
Component integration	Content projection, two-way binding, ARIA relationships, event handling.
Accessibility	axe-based checks plus keyboard interaction tests.
Visual regression	Every component variant in both themes and responsive widths.
End-to-end	Navigation, theme persistence, playground controls, clipboard success/failure.
Build	Standard and GitHub Pages builds with bundle budgets.

14. Security and privacy

- The app does not collect, transmit, or store personal data on a server.
- The only persistent browser value is the theme preference.
- No secrets or environment variables are required at runtime.
- No user-supplied HTML is inserted into the DOM; Angular interpolation escapes text.
- Clipboard writes occur only after explicit UI actions.
- Dependencies should still be reviewed and updated through normal supply-chain controls.

If network features, analytics, or authentication are added later, update the threat model, privacy notice, Content Security Policy, and deployment documentation.

15. Troubleshooting

Build reports an unsupported Node.js version

Install a Node release accepted by Angular CLI 22.1.8, then reinstall dependencies with `npm ci`.

Components render without styling

Add `ailabs-design-system/styles.css` to the consumer application's `angular.json`. Components depend on the packaged CSS variables and class rules.

Theme does not persist

Check whether the browser blocks local storage. The theme should still change for the current session even if persistence fails.

Copy buttons do not work

Run on localhost or HTTPS, allow clipboard access, and check browser developer tools. The current UI has no clipboard-error state.

GitHub Pages loads a blank or unstyled page

Use `npm run build:pages` and publish `dist/playground/browser/`. Confirm the repository/subpath casing exactly matches `/Ailabs-DesignSystem/`.

Calendar navigation does not change month

This is a known prototype limitation. The current calendar displays a fixed March grid and does not implement previous/next month logic.

Time selection does not update the parent form

`TimeClockComponent` currently changes only its internal inputs and exposes no outputs. Use `TimePickerComponent` for string entry or extend the clock with change events and a form-control contract.

16. Known limitations and roadmap priorities

The most important production-hardening gaps are:

1. No automated unit, accessibility, visual, or end-to-end test suite.
2. Date Calendar is fixed to a March sample and lacks real navigation, parsing, localization, and complete keyboard support.
3. Time Clock lacks outputs and complete input-mode behavior.
4. Tabs and menu items provide semantics/styles but not composite-widget keyboard management.
5. Search Panel displays demo results when its `results` input is empty and has no explicit loading or empty states.
6. Clipboard failures are not surfaced to users.
7. CSS and TypeScript token definitions are manually duplicated.
8. No formal supported-browser matrix or automated browser compatibility testing exists.
9. No license file, contribution guide, changelog, or release policy is present in the repository.

Recommended priority order: automated tests, date/time completion, accessibility hardening, token generation, then release governance.

17. Maintenance checklist

For every release or significant change:

- Update `package.json` version according to the release policy.
- Keep Angular packages on compatible versions.
- Run a clean install and both production build variants.
- Review output against bundle budgets.
- Verify component APIs and update this document.
- Check tokens in light and dark modes.
- Test keyboard and screen-reader behavior.
- Confirm the live demo base path and package stylesheet loading.
- Record breaking changes and migration notes.
- Publish only artifacts produced from a clean, reviewed commit.

18. Installation schematic and package contract

The ng-add schematic is registered in `schematics/collection.json` and implemented in `schematics/ng-add/index.cjs`. Its optional project string chooses one application project; without it, all application projects are considered. It reads `/angular.json`, supports `architect.build` or `targets.build`, and prepends the combined stylesheet to base build options. It recognizes existing string entries and objects whose input matches the stylesheet, avoiding duplicates.

It fails for a missing workspace file, an unknown/non-application target, or a workspace with no application projects. A project without build options is skipped with a warning. It does not modify TypeScript imports, inject components, add themes, update test targets, or handle configuration-specific style overrides. It parses JSON directly, so nonstandard JSON comments are not supported.

The library entry point is `src/public-api.ts`. Production compilation uses partial Angular compilation through `ng-packagr`. `ng-package.json` copies CSS into `styles` and the schematic folder into `schematics`. The generated manifest supplies Angular package exports; source package subpaths expose `styles.css`, `tokens.css`, and `components.css`. Inspect the generated manifest before publishing.

CSS is marked as a side effect in package metadata. `tslib` is a runtime dependency. Angular `common/core` are peers constrained to `>=22.0.0 <23.0.0`. The showcase additionally installs the `platform/browser/compiler` toolchain, `RxJS`, and `Zone.js`. There is no backend service or runtime environment file to configure.

19. Integration details and implementation boundaries

19.1 Form controls and state

The custom input, checkbox, switch, radio, date field, and time field components expose inputs and event emitters, not `ControlValueAccessor`. Direct `formControlName` and `ngModel` integration is not implemented. Use explicit bindings or build a forms adapter. Pass boolean inputs using property binding.

For radio groups, keep one selected value in the parent and update it from each option's `checkedChange`. Native same-name grouping manages browser selection, but it does not update every sibling component's input property or your parent model automatically.

```
<ds-radio name="model" value="fast" [checked]="model === 'fast'"
  (checkedChange)="model = 'fast'">Fast</ds-radio>
<ds-radio name="model" value="smart" [checked]="model === 'smart'"
  (checkedChange)="model = 'smart'">Smart</ds-radio>
```

Initialize `model = 'fast'` in the parent and import `RadioComponent`.

19.2 Search and toolbar composition

Search components provide local UI and outputs; they do not call an API, filter a remote data source, or provide speech recognition. Wire search, voice, back, and result selection to application behavior. `SearchBar` uses a form internally, so avoid nesting it inside another form.

Toolbar content is projected. Its variants do not implement dragging, viewport anchoring, or editor commands. Consumer buttons need handlers and labels. Roving keyboard focus for a complete composite toolbar is not implemented.

19.3 Browser and accessibility boundaries

The showcase accesses `window`, `document`, `local storage`, and `IntersectionObserver` directly; it has no SSR or hydration configuration. Tooltip descriptions are attached to a wrapper, so verify that the actual interactive trigger gets the intended accessible description. The checkbox models indeterminate appearance and `aria-checked`, but does not bind the native input's `indeterminate` property. Validate mixed-state behavior for the target assistive technologies.

The hero View on GitHub button has no handler; the header GitHub link is functional. Clipboard calls have no user-facing error handling. Theme persistence exists only in the showcase; consumers implement their own theme control.

20. Design-to-release operating model

Artifact	Source of truth	How it changes
Figma designs	Linked Figma source file	Edit main assets and publish library updates
Tokens	CSS values and TypeScript metadata	Manually update both and compare with design
Angular API	Component source and public barrel	Implement, review, build, package, and test
GitHub source	Reviewed commits on main	Branch, pull request, review, merge
Live showcase	Pages workflow artifact	Push to main or manual workflow dispatch
npm package	Built release archive	Explicit authenticated package publication
Consumer apps	Their source and lockfiles	Install a selected release and test migrations

No automatic Figma import/export pipeline, token generator, npm release workflow, or Code Connect mapping files are present. The README's synchronization description reflects intended design alignment, not an implemented sync service. Figma publication does not update CSS, and pushing GitHub source does not update npm consumers.

Use the companion Step-by-Step Guide for complete procedures with commands, permissions, expected results, and recovery. Its npm publication example builds and tests an archive from a reviewed commit, publishes that archive, and records the matching Git tag. Version numbers in release examples must be checked against the registry before use.

Appendix A. Public exports

The barrel file exports:

- BadgeComponent, BadgeTone
- ButtonComponent, ButtonSize, ButtonVariant
- CardComponent
- CheckboxComponent
- ChipComponent, ChipVariant
- DateCalendarComponent, DatePickerComponent, CalendarView
- InputComponent and alias TextFieldComponent
- ListItemComponent
- MenuItemComponent
- RadioComponent
- SearchBarComponent, SearchPanelComponent, SearchConfiguration, SearchResult, SearchState
- SwitchComponent
- TabComponent
- TimeClockComponent, TimePickerComponent, ClockMode, ClockView
- ToolbarComponent, ToolbarTone, ToolbarVariant

- TooltipComponent, TooltipPosition
- Token metadata: colorRamps, semanticColorGroups, spacing, radii, and strokes
- Token interfaces: ColorToken and ColorRamp

Appendix B. Configuration reference

Angular build inputs

Setting	Value
Browser entry	src/main.ts
Host page	index.html
TypeScript config	tsconfig.app.json
Inline style language	CSS
Static asset source	src/assets
Default build	Production
Default serve mode	Development

Global stylesheet order

1. src/design-system/tokens.css
2. src/design-system/components.css
3. src/styles.css

This order is intentional: token definitions precede component rules, and showcase-specific rules load last.

Document ownership: Keep this document synchronized with component source, package.json, and angular.json. If code and documentation disagree, the checked-in source code is the operational authority.

Appendix C. Verification and source references

The component APIs and operating instructions were reviewed against the local source at commit 88d2cc8. The library build completed successfully during documentation preparation. The Pages build also passed, producing a 307.72 kB initial bundle (75.15 kB estimated transfer). It required execution outside the restricted sandbox after sandboxed runs exited abnormally. These checks used the existing installed dependencies; a fresh npm ci and consumer installation were not run. No live Figma publication, GitHub push, npm publication, or fresh consumer installation was performed as part of this documentation task.

Primary repository references: `src/app/design-system/*.component.ts`, `src/app/design-system/index.ts`, `src/public-api.ts`, `src/app/app.component.*`, `src/design-system/tokens.*`, `src/design-system/components.css`, `package.json`, `angular.json`, `ng-package.json`, `tsconfig*.json`, `schematics/ng-add/*`, and `.github/workflows/deploy-pages.yml`.

External workflow sources, consulted 22 September 2026, are linked in the relevant sections and in the companion guide. Platform permissions and release availability should be rechecked at the time of use.